

Data Transformation

QUESTION 1

Handle Missing Values (Basic Cleaning)

Fill missing values using the following:

Age → Median

Salary → Mean

Hours_Worked_Per_Week → Median

Performance_Score → Mean

Display the dataset after handling missing values.

ANSWER

```
[43]: df['Age'].fillna(df['Age'].median(),inplace = True)
      #df.head(50)
      df['Salary'].fillna(df['Salary'].mean())
      #df
      df["Hours_Worked_Per_Week"].fillna(
          df["Hours_Worked_Per_Week"].median())
      df
      df["Performance_Score"].fillna(
          df["Performance_Score"].mean())
      df
      print(df)
```

	Employee_ID	Name	Department	Age	Gender	Work_Mode	Join_Date	\
0	E001	Amit Sharma	IT	25.0	Male	Remote	15-01-2021	
1	E002	Riya Patel	HR	30.0	Female	Hybrid	15-02-2020	
2	E003	John Mathew	Finance	29.0	Male	Onsite	10-03-2022	
3	E004	Sneha Joshi	IT	28.0	Female	Remote	22-04-2021	
4	E005	Arjun Verma	Marketing	35.0	Male	Hybrid	18-05-2020	
..	...	...	...	...	...	...	...	...
245	E006	Neha Singh	HR	27.0	Female	Remote	01-06-2023	
246	E007	Rahul Das	Finance	40.0	Male	Onsite	07-07-2019	
247	E008	Pooja Mehta	Marketing	29.0	Female	Hybrid	12-08-2022	
248	E009	Sameer Khan	IT	31.0	Male	Remote	30-09-2021	
249	E010	Kiran Nair	HR	26.0	Female	Onsite	05-10-2023	

	Hours_Worked_Per_Week	Projects_Completed	Performance_Score	\
0	40.0	5	4.5000	
1	38.0	4	4.0000	
2	45.0	6	4.3125	
3	41.0	5	4.2000	
4	50.0	7	4.8000	
..	...	...	...	...
245	36.0	3	3.9000	
246	48.0	8	4.7000	
247	42.0	6	4.3000	
248	41.0	5	4.1000	
249	37.0	4	4.3125	

QUESTION 2

Label Encoding

Convert these columns into numeric values using Label Encoding:

Gender

Department

Show the updated columns.

ANSWER

```
61]: # LABEL ENCODER
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Gender"] = le.fit_transform(df["Gender"])
df["Department"] = le.fit_transform(df["Department"])
print(df[["Gender", "Department"]])
```

	Gender	Department
0	1	2
1	0	1
2	1	0
3	0	2
4	1	3
..	...	...
245	0	1
246	1	0
247	0	3
248	1	2
249	0	1

[250 rows x 2 columns]

QUESTION 3

One-Hot Encoding

Apply One-Hot Encoding on:

Work_Mode

Location

ANSWER

Display the dataset and check how many new columns are created.

```
[64]: #Apply One-Hot Encoding on:
df_encoded = pd.get_dummies(df, columns=['Work_Mode', 'Location'])
print(df_encoded)
```

Employee_ID	Name	Department	Age	Gender	Join_Date	
0	E001	Amit Sharma	2	25.0	1	15-01-2021
1	E002	Riya Patel	1	30.0	0	15-02-2020
2	E003	John Mathew	0	29.0	1	10-03-2022
3	E004	Sneha Joshi	2	28.0	0	22-04-2021
4	E005	Arjun Verma	3	35.0	1	18-05-2020
...	...	...	...	...	...	...
245	E006	Neha Singh	1	27.0	0	01-06-2023
246	E007	Rahul Das	0	40.0	1	07-07-2019
247	E008	Pooja Mehta	3	29.0	0	12-08-2022
248	E009	Sameer Khan	2	31.0	1	30-09-2021
249	E010	Kiran Nair	1	26.0	0	05-10-2023

Hours_Worked_Per_Week	Projects_Completed	Performance_Score	
0	40.0	5	4.5000
1	38.0	4	4.0000
2	45.0	6	4.3125
3	41.0	5	4.2000
4	50.0	7	4.8000
...	...	...	...
245	36.0	3	3.9000
246	48.0	8	4.7000
247	42.0	6	4.3000
248	41.0	5	4.1000
249	37.0	4	4.3125

Salary	Work_Mode_Hybrid	Work_Mode_Onsite	Work_Mode_Remote	
0	50000.000000	False	False	True
1	45000.000000	True	False	False
2	60000.000000	False	True	False
3	52000.000000	False	False	True
4	50666.666667	True	False	False
...	...	...	...	...
245	42000.000000	False	False	True
246	65000.000000	False	True	False
247	48000.000000	True	False	False
248	51000.000000	False	False	True
249	43000.000000	False	True	False

Location_Bangalore	Location_Delhi	Location_Mumbai	Location_Pune	
0	False	False	False	True
1	False	False	True	False
2	True	False	False	False
3	False	True	False	False
4	False	False	False	True
...	...	...	...	...
245	False	False	True	False

QUESTION 4

Normalization (Min-Max Scaling)

Normalize the following columns:

Salary

Hours_Worked_Per_Week

Use MinMaxScaler and display results.

ANSWER

```
[81]: #Normalization (Min-Max Scaling)
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
#df[['Salary_Scaled']] = scaler.fit_transform(df[['Salary', 'Hours_Worked_Per_Week']])
df[["Salary", "Hours_Worked_Per_Week"]] = scaler.fit_transform(
    df[["Salary", "Hours_Worked_Per_Week"]]
)
print(df[['Salary', 'Hours_Worked_Per_Week']])
```

	Salary	Hours_Worked_Per_Week
0	0.347826	0.285714
1	0.130435	0.142857
2	0.782609	0.642857
3	0.434783	0.357143
4	0.376812	1.000000
..	...	...
245	0.000000	0.000000
246	1.000000	0.857143
247	0.260870	0.428571
248	0.391304	0.357143
249	0.043478	0.071429

[250 rows x 2 columns]

QUESTION 5

Standardization (Scaling)

Apply StandardScaler on:

ANSWER

AgeProjects_Completed

Display the transformed values.

The screenshot shows a Jupyter Notebook interface with a browser window at localhost:8888. The notebook contains a code cell with the following Python code:

```
[83]: df[["Age", "Projects_Completed"]] = scaler.fit_transform(
      df[["Age", "Projects_Completed"]])
      print(df[["Age", "Projects_Completed"]])
```

Below the code cell, the output of the code is displayed as a table with 250 rows and 2 columns. The first few rows are shown, followed by an ellipsis, and then the last few rows.

	Age	Projects_Completed
0	0.000000	0.4
1	0.333333	0.2
2	0.266667	0.6
3	0.200000	0.4
4	0.666667	0.8
..	...	...
245	0.133333	0.0
246	1.000000	1.0
247	0.266667	0.6
248	0.400000	0.4
249	0.066667	0.2

[250 rows x 2 columns]

QUESTION 6

Compare Scaling Methods

Apply both:

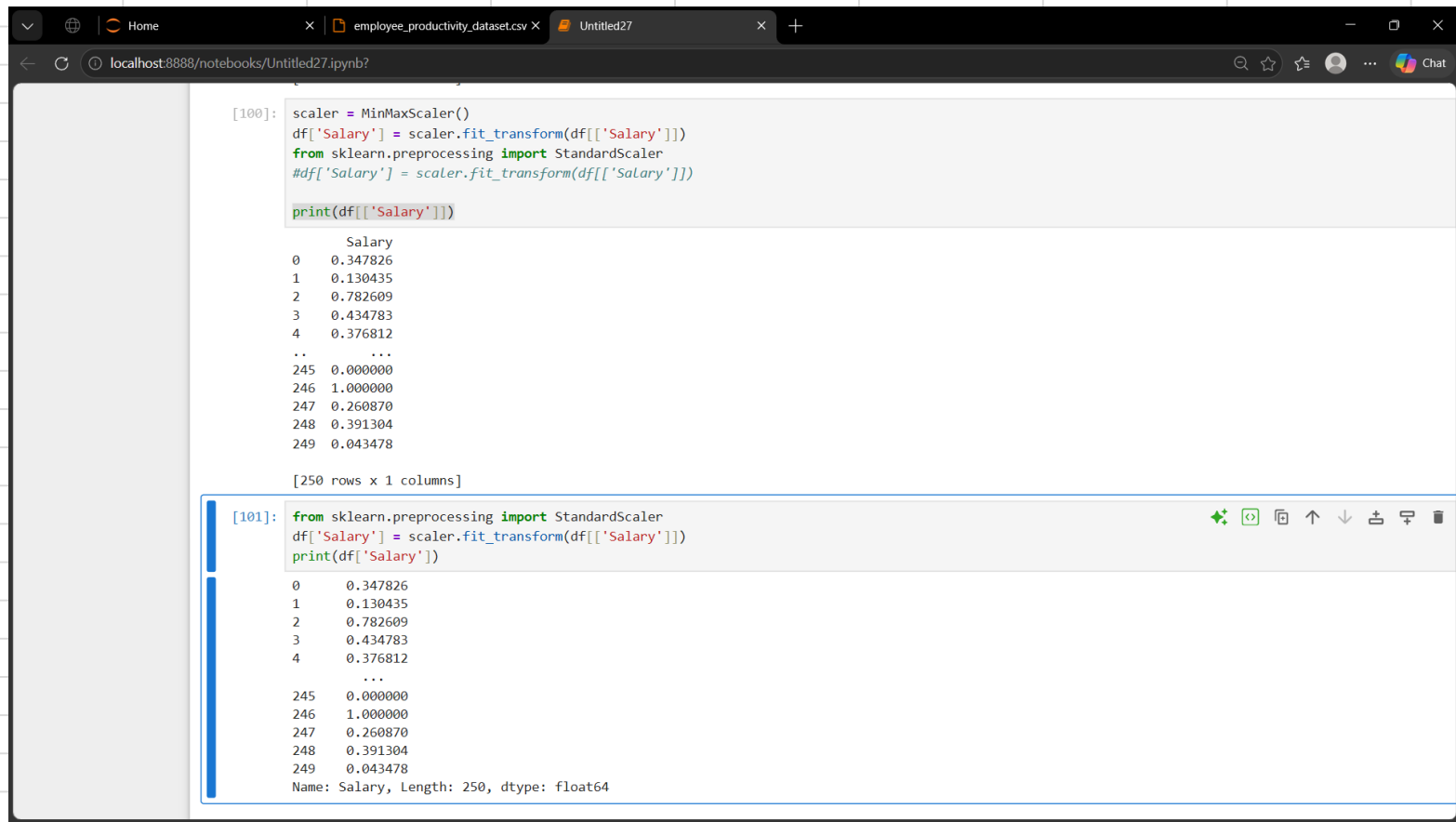
MinMaxScaler

StandardScaler

On the Salary column.

Show both results side by side.

ANSWER



```
[100]: scaler = MinMaxScaler()
df['Salary'] = scaler.fit_transform(df[['Salary']])
from sklearn.preprocessing import StandardScaler
#df['Salary'] = scaler.fit_transform(df[['Salary']])

print(df[['Salary']])

Salary
0    0.347826
1    0.130435
2    0.782609
3    0.434783
4    0.376812
...
245  0.000000
246  1.000000
247  0.260870
248  0.391304
249  0.043478

[250 rows x 1 columns]
```

```
[101]: from sklearn.preprocessing import StandardScaler
df['Salary'] = scaler.fit_transform(df[['Salary']])
print(df['Salary'])

0    0.347826
1    0.130435
2    0.782609
3    0.434783
4    0.376812
...
245  0.000000
246  1.000000
247  0.260870
248  0.391304
249  0.043478
Name: Salary, Length: 250, dtype: float64
```

QUESTION 7	Create a pipeline that:				
	Applies encoding to categorical columns				
	Applies scaling to numerical columns				
	Use:				
	ColumnTransformer				
	Pipeline				

ANSWER

```
•[107]: from sklearn.compose import ColumnTransformer
        from sklearn.preprocessing import OneHotEncoder
        from sklearn.pipeline import Pipeline

        categorical_cols = [
            "Gender",
            "Department",
            "Work_Mode",
            "Location"
        ]
        numerical_cols = [
            "Age",
            "Salary",
            "Hours_Worked_Per_Week",
            "Performance_Score",
            "Projects_Completed"
        ]
        preprocessor = ColumnTransformer(
            transformers=[
                (
                    "num",
                    StandardScaler(),
                    numerical_cols
                ),
                (
                    "cat",
                    OneHotEncoder(handle_unknown="ignore"),
                    categorical_cols
                )
            ]
        )
        pipeline = Pipeline(
            steps=[
                ("preprocessor", preprocessor)
            ]
        )
        processed_data = pipeline.fit_transform(df)
        print("Processed Data Shape:", processed_data.shape)
        print(processed_data)

Processed Data Shape: (250, 18)
[[-1.17201808e+00 -9.74508910e-02 -4.11220496e-01 ... 0.00000000e+00
  0.00000000e+00 1.00000000e+00]
 [-1.95180218e-16 -8.28332574e-01 -8.68132159e-01 ... 0.00000000e+00
  1.00000000e+00 0.00000000e+00]
 [-2.34403615e-01 1.36431247e+00 7.31058660e-01 ... 0.00000000e+00
  0.00000000e+00 0.00000000e+00]
 ...
 [-2.34403615e-01 -3.89803564e-01 4.56911662e-02 ... 0.00000000e+00
  0.00000000e+00 0.00000000e+00]
 [ 2.34403615e-01 4.87254455e-02 -1.82764665e-01 ... 0.00000000e+00
  0.00000000e+00 0.00000000e+00]
```

QUESTION 8

Apply Pipeline

Apply the pipeline on the dataset.

Display:

Transformed dataset

ANSWER

Shape of final dataset

The screenshot shows a Jupyter Notebook window with two tabs: 'employee_productivity_dataset.csv' and 'Untitled27'. The browser address bar shows 'localhost:8888/notebooks/Untitled27.ipynb?'. The notebook interface has a left sidebar and a main area. The main area displays a code cell with a red error message at the top and a successful execution cell below it.

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[109], line 5  
      3 from sklearn.pipeline import Pipeline  
      4 print("Shape of Final Dataset:")  
----> 5 print(transformed_data.shape)  
  
NameError: name 'transformed_data' is not defined
```

```
[112]: transformed_data = pipeline.fit_transform(df)  
print("Shape of Final Dataset:")  
print(transformed_data.shape)  
print("\nTransformed Dataset:")  
print(transformed_data)
```

Shape of Final Dataset:
(250, 18)

Transformed Dataset:
[[-1.17201808e+00 -9.74508910e-02 -4.11220496e-01 ... 0.00000000e+00
 0.00000000e+00 1.00000000e+00]
 [-1.95180218e-16 -8.28332574e-01 -8.68132159e-01 ... 0.00000000e+00
 1.00000000e+00 0.00000000e+00]
 [-2.34403615e-01 1.36431247e+00 7.31058660e-01 ... 0.00000000e+00
 0.00000000e+00 0.00000000e+00]
 ...
 [-2.34403615e-01 -3.89803564e-01 4.56911662e-02 ... 0.00000000e+00
 0.00000000e+00 0.00000000e+00]
 [2.34403615e-01 4.87254455e-02 -1.82764665e-01 ... 0.00000000e+00
 0.00000000e+00 1.00000000e+00]
 [-9.37614462e-01 -1.12068525e+00 -1.09658799e+00 ... 0.00000000e+00
 1.00000000e+00 0.00000000e+00]]

QUESTION 9	Conceptual Question Why is scaling important in Python?
ANSWER	Importance of Scaling 1. Ensures Fair Comparison Scaling puts all features on a similar scale, so no feature dominates because of its larger values. 2. Improves Model Performance Many machine learning algorithms perform better when the data is scaled. 3. Speeds Up Training Scaled data helps models learn faster and converge more quickly. 4. Important for Distance-Based Algorithms Algorithms such as: K-Nearest Neighbors (KNN) K-Means Support Vector Machines (SVM)
	Common Types of Scaling Min-Max Scaling Transforms values into a range between 0 and 1. Standardization (StandardScaler) Transforms data so that: Mean = 0 Standard Deviation = 1

QUESTION 10	Conceptual Question Why do we convert categorical data into numerical form?
ANSWER	Most machine learning algorithms cannot understand text or categorical values directly. They work only with numbers. A machine learning model cannot perform calculations on "Male" and "Female". Therefore, we convert them into numbers. Reasons for Converting Categorical Data - Machine Learning Models Require Numbers Algorithms perform mathematical operations, so categorical data must be converted into numerical form. - Enables Data Processing Numerical data can be analyzed, scaled, and used for predictions. - Improves Model Performance Proper encoding helps the model understand patterns in the data. We convert categorical data into numerical form because machine learning algorithms can only work with numerical values. Encoding techniques such as Label Encoding and One-Hot Encoding help transform categorical data into a format that models can understand and process